

KnowledgeBasev3

Developer Documentation

Architecture • API Reference • Data Model • Best Practices

Version 1.0 | Confidential

1. Purpose

This document is the primary reference for developers working on KnowledgeBasev3. It covers architecture, development workflow, environment variables, key commands, API surface, data model, and recommended best practices. Keep this document updated whenever the API contract or architecture changes significantly.

2. Quick Start (Local)

2.1 Prerequisites

- Node.js >= 18 (required)
- PostgreSQL (optional — only needed for DB-backed mode)

2.2 Installation

```
| npm ci
```

Or alternatively:

```
| npm install
```

2.3 Environment Setup

Create a .env.local file in the project root. See Section 3 for the full list of required and optional environment variables.

2.4 Starting the Development Server

```
| npm run dev           # Runs: tsx server.ts (Express + Vite middleware)
```

2.5 Building & Production

```
| npm run build          # Frontend build via Vite npm run start      #  
| Production server (after build)
```

i Note: Docker is also supported. Run `docker-compose up --build` to start the app alongside a Postgres service as defined in the repo.

3. NPM Scripts Reference

Script	Description
dev	tsx server.ts — dev server with Vite HMR middleware
start	node server.ts — run server in Node (production)
build	vite build — compile and bundle the frontend
preview	vite preview — preview production build locally
db:push	drizzle-kit push — apply Drizzle schema to the database
db:studio	drizzle-kit studio — open Drizzle Studio GUI
lint	tsc --noEmit — TypeScript type checking

4. Environment Variables

Variable	Description
DATABASE_URL	PostgreSQL connection string. If absent, the server falls back to in-memory mock data (no DB required for local dev).
GEMINI_API_KEY	API key for client-side AI integrations (@google/genai). Required only when using AI features.
PGSSLMODE	Optional. Set to require when using cloud hosts (e.g. Neon). Enables SSL with rejectUnauthorized: false on the DB pool.

Note: Never commit secrets to version control. Use `.env.local` for local development and CI secret stores for deployment pipelines.

5. Database & Migrations

5.1 ORM

The project uses Drizzle ORM. Schema definitions live in `src/db/schema.ts`; configuration is in `drizzle.config.ts`.

5.2 Key Commands

```
npm run db:push      # Push schema changes to the database
npm run db:studio    # Open Drizzle Studio to inspect data
```

5.3 Connection

src/db/index.ts creates a pg.Pool and Drizzle client. SSL is enabled automatically when DATABASE_URL contains .neon. or PGSSLMODE is set to require.

5.4 Mock / In-Memory Mode

When DATABASE_URL is not set, the server falls back to mockArticles, mockCategories, and inMemoryLogs. This allows running the app locally without any database setup. The server also auto-seeds mock data when the database tables are empty.

6. Data Model

6.1 users

Column	Description
id	Primary key
email	Unique user email address
password	bcryptjs-hashed password
name	Display name
role	User role (e.g. admin, viewer)
apiKey	Bearer token for public API access (prefix: ied_)
createdAt	Timestamp of account creation

6.2 categories

Column	Description
id	Primary key
title	Display title
filterCategory	Filter slug
icon	Icon identifier
count	Article count
colorClass	CSS color class
group	Category group

6.3 articles

Column	Description
id	Primary key
title	Article title
excerpt	Short summary
content	Full article body
author	Author name
date	Publication date
views	View count
category	Associated category
categoryColor	Color for category badge
badge	Display badge label
status	Draft / published / archived
accessLevel	Visibility access level
attachments	Attached file references
createdAt	Record creation timestamp

6.4 system_logs

Column	Description
id	Primary key
level	Log level (info, warn, error)
message	Log message text
date	Log date (YYYY-MM-DD)
createdAt	Exact timestamp

7. API Reference

All server APIs are hosted under the /api prefix.

7.1 Authentication

Method & Endpoint	Description
POST /api/auth/signup	Create a new user account. Password is hashed with bcryptjs.
POST /api/auth/signin	Authenticate user. Returns user profile and API key.
POST /api/auth/generate-key	Generate a new API key for a user (prefix: ied_).

7.2 Articles

Method & Endpoint	Description
GET /api/articles	List all articles (DB or in-memory).
POST /api/articles	Create a new article.
PUT /api/articles/:id	Update an existing article by ID.
DELETE /api/articles/:id	Delete an article by ID.

7.3 Categories

Method & Endpoint	Description
GET /api/categories	List all categories. Auto-seeds from mock data if empty.
POST /api/categories	Create a new category.

7.4 Users

Method & Endpoint	Description
GET /api/users	List all users (DB-only).
PUT /api/users/:id/role	Update a user's role.
DELETE /api/users/:id	Delete a user.

7.5 System Logs

Method & Endpoint	Description
GET /api/logs?date=YYYY-MM-DD	Retrieve log entries for a specific date.
POST /api/logs	Create a new log entry (DB or in-memory).

7.6 Public API (Integration)

Namespace: /api/public/v1/*

i Note: When `DATABASE_URL` is set, all public API routes require authentication via the `x-api-key` request header or the `apiKey` query parameter. Keys are stored in `users.api_key` and always begin with `ied_`.

8. Frontend Architecture

8.1 Auth State

`src/AuthContext.tsx` manages authentication state and persists the logged-in user to `localStorage` under the key `ied_user`.

8.2 App Entry Point

`src/App.tsx` is the top-level client component. It handles routing, data fetching from `/api/articles` and `/api/categories`, and role-based visibility logic.

8.3 Directory Structure

Path	Purpose
<code>src/views/</code>	Top-level page views
<code>src/components/</code>	Reusable UI components
<code>src/AuthContext.tsx</code>	Auth state management and <code>localStorage</code> binding
<code>src/App.tsx</code>	Routing, data fetching, role-based UI

8.4 AI Features

AI features use the `@google/genai` SDK and require `GEMINI_API_KEY` to be set in the environment.

9. Key File Locations

File	Purpose
<code>server.ts</code>	Backend entry point

File	Purpose
src/db/schema.ts	Drizzle ORM schema definitions
src/db/index.ts	DB pool and Drizzle client initialization
drizzle.config.ts	Drizzle configuration file
src/App.tsx	Frontend top-level component
src/AuthContext.tsx	Auth state and localStorage binding
package.json	Scripts and dependency manifest
TECHNICAL_SPEC.txt	Detailed project specification

10. Development & Debugging

10.1 Recommended Workflow

- Use `npm run dev` for local development — it serves both Express and the Vite frontend with hot module replacement.
- Use `npm run lint (tsc --noEmit)` regularly to catch type errors early.
- When the database is empty, the server auto-seeds `mockArticles` and `mockCategories` on first request.

10.2 Database Connection Issues

If the database connection fails, verify the following:

- `DATABASE_URL` is correctly set and reachable.
- `PGSSLMODE` is set to `require` if using a cloud host.
- Network/firewall access to the Postgres server is open.
- Review SSL handling logic in `src/db/index.ts`.

10.3 Viewing Logs

Check the server console for runtime output. You can also query log entries via the API:

```
| GET /api/logs?date=YYYY-MM-DD
```

11. Security Notes

⚠ Warning: *Server-side RBAC enforcement is minimal in the current implementation. Role checks are performed client-side for UI visibility only. Before deploying to production, implement server-side authorization checks on all sensitive endpoints.*

11.1 Password Storage

All user passwords are hashed using bcryptjs at signup. Plaintext passwords are never stored.

11.2 API Keys

API keys are simple bearer tokens stored in the users.api_key database column. Keys are prefixed with ied_ for easy identification. Treat them as secrets — rotate immediately if compromised.

11.3 Secrets Management

- Never commit secrets or API keys to version control.
- Use .env.local for local development (this file should be in .gitignore).
- Use CI/CD secret stores (e.g. GitHub Actions Secrets, Vault) for deployment pipelines.

12. Code Quality & Best Practices

12.1 TypeScript

The project is fully typed. Keep type errors at zero by running npm run lint before committing. Never suppress type errors with any casts unless absolutely necessary and documented.

12.2 Version Control

- Prefer small, focused commits over large catch-all commits.
- Write descriptive commit messages (what changed and why).
- Use feature branches; avoid committing directly to main.

12.3 Testing

- Add unit and integration tests for new features and bug fixes.
- Test both DB-backed and in-memory (mock) modes.

12.4 Documentation

- Document any new API endpoints or contract changes in this file.
- Update TECHNICAL_SPEC.txt for significant architectural decisions.